Proof of Correctness of Atomic Load, Store, and CAS Timestamping Algorithm

Soo-Ho Park
spark108@student.ubc.ca

1. Algorithm

The original algorithm has been translated into C++ with a few modifications. First, a Clock class was introduced to keep track of the global timestamp that all TimestampedAtomic objects depend on. Advancement of the global timestamp occurs exclusively through invocations of `get_timestamp()`. Next, all CAS operations have been translated to either `compare_exchange_weak` or `compare_exchange_strong` depending on whether the CAS is allowed to try again. Consequently, a local variable was introduced in `help_timestamp()` because `compare_exchange_strong` updates the expected value to the current value upon failure. The memory leak in `store()` has been fixed by eliminating the allocation of a new node on each failed CAS attempt. Most importantly, a logical error in `CAS()` has been addressed. Previously, when the `CAS()` operation failed, the function returned the timestamp of the most recently stored value, i.e. the time at which that value was successfully inserted into the linked list. This behavior has been corrected so that, on failure, the function instead returns the timestamp associated with the failed CAS attempt itself. This change ensures that the linearization point of a failed CAS is well-defined in the proof presented later.

The specifications of the three operations are as follows:

- load(): Returns the value of the TimestampedAtomic object. More precisely, it returns the value that the most recent store() or modifying CAS() stored into the object before the linearization point of itself. The timestamp returned by this operation is a new timestamp it reads from the global Clock object when it starts its execution.

- store(T value): Stores a new value into the TimestampedAtomic object and returns the timestamp corresponding to the completion of its installation.

- CAS(T expected, T desired): If the current value of the TimestampedAtomic object is not equivalent to the given expected value, it returns false with the new timestamp it read from the global Clock object when it started its execution. If the current value is equivalent to the given expected value, it stores the given desired value into the TimestampedAtomic object. Then, it returns true and the timestamp corresponding to the completion of the installation of the new value.

Clock class

```
class Clock {
    private:
        std::atomic<int> timestamp;
    public:
        Clock() { timestamp = 0; }
        int get_timestamp() {
            return timestamp.fetch_add(1);
        }
};
```

TimestampedAtomic class

```
template <typename T>
class TimestampedAtomic {
    private:
        struct Node {
            T val;
            std::atomic<int> timestamp;
            Node* prev;
        };
        std::atomic<Node*> head;
        Clock* clock;
        const int TBD = -1;
        void help_timestamp(Node* node);
    public:
        TimestampedAtomic(T val, Clock* c);
        std::pair<T, int> load();
        T load_no_timestamping();
        int store(T newVal);
        std::pair<bool, int> CAS(T expected, T desired);
};
```

Implementation of TimestampedAtomic

```cpp
template <typename T>
TimestampedAtomic<T>::TimestampedAtomic(T val, Clock* c)
{   clock = c;
    head = new Node{val, TBD, nullptr};
    help_timestamp(head.load());
}
```

```cpp
template <typename T>
void TimestampedAtomic<T>::help_timestamp(typename TimestampedAtomic<T>::Node* node) {
    if (node != nullptr && node -> timestamp.load() == TBD) {
        int expected = TBD;
        node -> timestamp.compare_exchange_strong(expected, clock -> get_timestamp());
    }
}
```

```cpp
template <typename T>
std::pair<T, int> TimestampedAtomic<T>::load() {
    int read_ts = clock -> get_timestamp();
    Node* curr = head.load();
    help_timestamp(curr);

    while (curr != nullptr && curr -> timestamp.load() > read_ts) {
        curr = curr -> prev;
    }

    return {curr -> val, read_ts};
}
```

```cpp
template <typename T>
int TimestampedAtomic<T>::store(T newVal) {
    Node* newNode = new Node {newVal, TBD, nullptr};
    while(true) {
        Node* curr = head.load();
        help_timestamp(curr);

        newNode -> prev = curr;

        if (head.compare_exchange_weak(curr, newNode)) {
            help_timestamp(newNode);
            return newNode -> timestamp.load();
        }
    }
}
```

```cpp
template <typename T>
std::pair<bool, int> TimestampedAtomic<T>::CAS(T expected, T desired) {
    Node* newNode = new Node{desired, TBD};
    while(true) {
        int read_ts = clock -> get_timestamp();
        Node* curr = head.load();
        help_timestamp(curr);

        while (curr != nullptr && curr -> timestamp.load() > read_ts) {
            curr = curr -> prev;
        }

        if (curr -> val != expected) {
            delete newNode;
            return {false, read_ts};
        }

        newNode -> prev = curr;

        if (head.compare_exchange_weak(curr, newNode)) {
            help_timestamp(newNode);
            return {true, newNode -> timestamp.load()};
        }
    }
}
```

2. Linearization Points

- `load()`: Line 3 when the function reads the global timestamp using `get_timestamp()`

  The purpose of `load()` is to return a node associated with the timestamp obtained from `get_timestamp()`. Hence, this may not be the most recent node in the linked list because other threads may have inserted more nodes into the list before `load()` returns.

  Suppose the linearization point was at Line 11 when the function returns. This implies that the function returns the most recent node in the list. Consider two threads A and B such that A performs a load operation and B performs a store operation.

  Assume the following timeline of instructions:

  A: `ts = get_timestamp()`

  B: start `store()`

  B: insert new node with value X

  B: end `store()` - *linearized somewhere between start and end* `store()`

  A: return node with timestamp at most ts - *linearization point*

  The sequence of operations is linearized as (B, A). Timestamp ts is smaller than the timestamp set for the node with value X. Thus, A will skip the node with value X and return a node farther down the list. This indicates that the store operation happened after the load operation. This does not agree with the linearized sequence of operations (B, A) which implies that A would return the node with value X. This is a counterexample for why `load()` cannot be linearized when the function returns.

- `store()`: After a node is inserted into the list at Line 10, the thread attempts to set the timestamp of that node. If it succeeds, then the `get_timestamp()` before the CAS operation in `help_timestamp()`. Otherwise, the `get_timestamp()` in `help_timestamp()` performed by whichever thread succeeds in setting the timestamp.

  The `store()` operation is linearized when the timestamp is decided for the newly inserted node. A node cannot be traversed in a `load()` operation before its timestamp has been set (in this case, `load()` will set the timestamp instead). Thus, the node is only visible to other threads after the timestamp is set.

  Suppose the linearization point was at Line 10 when the CAS succeeds in appending the newly created node. Consider three threads A and B such that A performs `store()` with the newly created node with value X and B performs `load()`.

  Assume the following timeline of instructions:

A: successfully inserts node with value X - *linearization point*

B: `ts = get_timestamp()` - *linearization point*

B: sets timestamp for node with value X using `help_timestamp()`

A: fails CAS in `help_timestamp()`

B: return node with timestamp at most ts

The sequence of operations is linearized as (A, B). The timestamp returned from `get_timestamp()` is a smaller value than the timestamp it set for the node with value X using `help_timestamp()`, since it was performed earlier. Thus, B will skip the node with value X when traversing the list and return a node farther down the list. This logically implies that the load operation happened before the store operation. Conversely, according to the sequence of linearization points (A, B), B should have returned the node with value X that A stored. Thus, this is a counterexample why the linearization point for `store()` cannot be when the node is successfully inserted into the list.

Suppose the linearization point for a store operation is at the moment the timestamp is set for the newly created node by whichever thread succeeds in doing it. There is a gap between retrieving the global timestamp and setting that same timestamp into the newly created node. If there were an intervening load operation, it may read the value in that node even though its linearization point is after that of the load operation. As a concrete example, consider two threads A and B that perform `store()` and `load()` respectively.

A: `help_timestamp()` after succeeding in appending newly created Node N

A: `get_timestamp()` in `help_timestamp()` returns timestamp `ts`

B: `ts = get_timestamp()` - *linearization point*

A: `node->timestamp.compare_exchange_strong(expected, ts)`

- *linearization point*

B: return value V in N

The sequence of operations is linearized as (B, A). In this case, B is able to read the value of Node N even though the linearized sequence states that the load operation (thread B) happened before the store operation (thread A). Since it is clearly incorrect for a load operation to observe a value that has not yet been inserted into the data structure, placing the linearization point at the moment the timestamp is assigned to the newly created node is invalid.

- Failed `CAS()`: Line 5 when the function reads the global timestamp using `get_timestamp()`

  Similar to `load()`, the operation is linearized when the function reads the global timestamp. A CAS operation fails when the current value does not match the expected value. It can thus be viewed as a `load()` operation augmented with an equality check. Thus, the linearization point is equivalent to the case of `load()`.

- Successful `CAS()`: After a node is inserted into the list at Line 20, the thread attempts to set the timestamp of that node. If it succeeds, then the `get_timestamp()` before the CAS operation in `help_timestamp()`. Otherwise, the `get_timestamp()` in `help_timestamp()` performed by whichever thread succeeds in setting the timestamp.

  Similar to `store()`, the operation is linearized when the timestamp for the newly created node is decided. The node is only visible i.e. able to be returned from subsequent `load()` operations when its timestamp is valid. Thus, the linearization point is equivalent to the case of `store()`.

  The linearization point cannot occur at the instant the node is inserted into the list. The counterexample for this is equivalent to the one found in the description of `store()`'s linearization point.

3. Proof of Correctness

In this section, we prove that the atomic timestamping algorithm is linearizable. We first provide some lemmas that will aid in our formal proof of linearizability.

A timestamp is considered valid if it is not TBD; otherwise, it is considered invalid.

Lemma 1. After an execution of help_timestamp on a given node, that node has a valid timestamp. Furthermore, help_timestamp is a wait-free operation.

Proof. The help_timestamp() function enters the if clause when the given node has an invalid timestamp. Therefore, if execution does not enter the if clause, the node must have a valid timestamp.If execution enters the if clause and the CAS succeeds, the current thread successfully updates the timestamp to a valid value. If the CAS fails, the current value of the timestamp must differ from TBD, implying that another thread has already set the timestamp to a valid value. Thus, in all executions of help_timestamp(), the given node has a valid timestamp upon termination, and the function completes in a constant number of

instructions. Moreover, a node's timestamp is modified only within the help_timestamp function. Since this function updates the timestamp only if it is TBD, a valid timestamp assigned by the successful help_timestamp invocation remains unchanged. ∎

Lemma 2. The only node in the TimestampedAtomic object's linked list that can have an invalid timestamp is the very first node of the list.

Proof. In the implementation of load(), store(), and CAS(), there is no assignment to the value of a Node except for the initial construction of it. help_timestamp() is the only operation that potentially changes the timestamp of a Node, but at Line 3, the if-clause is only entered when the current timestamp is equal to TBD. Thus, if a valid timestamp is set for a Node, it is not manipulated further. A Node is only deleted at Line 14 of CAS(). A Node is physically inserted into the list if the condition at Line 20 is true. If the condition is met and enter the if-clause, we return true and exit the function at Line 22. Thus, if a node is successfully inserted into the list, it cannot be deleted at Line 14, as the execution never reaches that line. In conclusion, a node is never deleted nor manipulated once it has been inserted into the shared list.

The only case where the list is changed is when we have a new node appended to the head of the list. This change can only be produced by store() and modifying CAS() operations. Both operations successfully append a new node after the successful execution of the CAS on the head pointer at Lines 10 and 18, respectively. Both operations call help_timestamp() on the current head node on Line 6 before the CAS. By Lemma 1, the head node has a valid timestamp after Line 6. Therefore, by induction, every newly appended node is added to a list consisting entirely of nodes with valid timestamps. ∎

Lemma 3. get_timestamp() invocations ordered by real-time return timestamps in strictly increasing order.

Proof. Every TimestampedAtomic object is associated with a single shared Clock object, which contains a single atomic integer field, timestamp. The only operation that modifies this field is get_timestamp(). Since the field is incremented using an atomic fetch_add operation, every invocation increments the variable exactly once and returns a unique value. Thus, the returned values are strictly increasing. ∎

Lemma 4. The shared list is composed of nodes in the order that they have been inserted.

Proof. At Line 10 of the store() operation, a thread attempts to insert the node it has created into the list. This is implemented using a compare_exchange_weak instruction, implying that at most one thread can successfully update the shared head pointer at any given time. Upon a successful compare_exchange_weak operation, the head pointer is updated to point to the

newly created node, which in turn points to the previous head node. The newly created node therefore becomes the new head of the list. Since the compare_exchange_weak operation can succeed for at most one thread at a time, at most one node can be inserted into the shared list at a time. Consequently, the shared list is composed of nodes in the order that they were successfully inserted into the list. ∎

The previous lemma establishes that the shared list consists of nodes ordered according to their insertion order. We define the **insertion time** of a node as the instant at which the node is successfully inserted into the list.

Lemma 5. The get_timestamp() invocations associated with the nodes in the list are totally ordered by insertion time, inducing a strictly decreasing order of timestamps in the list at all times (except possible at the head node, which may temporarily be TBD).

Proof. By Lemma 4, the shared list is composed of nodes in the order that they have been inserted. After a successful insertion of a node, the node is now part of the shared list and is visible to other threads. The thread that inserted the node attempts to set its timestamp at either Line 11 of store() or Line 21 of a successful CAS(), via a help_timestamp() invocation that in turn calls get_timestamp(). Other threads can set the node's timestamp first, after the head pointer has been read, through a help_timestamp() invocation at Line 5 of load(), Line 6 of store(), or Line 7 of CAS(). By Lemma 1, concurrent invocations of help_timestamp() each terminate after a finite number of instructions and collectively set the node's timestamp correctly. In any case, help_timestamp() is called on a node that is already part of the shared list and thus, get_timestamp() can only be called on an inserted node.

By Lemma 2, another node can be appended to the list only after the current head node has a valid timestamp, since at most one node in the list can be invalid at any given time. Therefore, there must be a get_timestamp() invocation and its corresponding timestamp installation before another node can be inserted. Thus, in the history of the shared list, the get_timestamp() invocations of each node are totally ordered, with the head node corresponding to the most recent invocation. This ordering aligns with when each node was inserted into the list. By Lemma 3, totally ordered invocations of get_timestamp() return strictly increasing timestamps. Consequently, the list maintains a strictly decreasing order of timestamps. The head node may temporarily have an invalid timestamp when it is inserted, but it will eventually receive a timestamp strictly larger than that of its successor, thereby preserving the strictly decreasing order of timestamps. ∎

Lemma 6. The timestamp stored in each node is identical to the timestamp returned by the corresponding store() operation that inserted the node into the list.

Proof. Each store() operation creates a node containing the specified value at Line 3. It first updates the new node to point to the current head node at Line 8, and then attempts to atomically update the head pointer to point to the newly created node at Line 10. If the attempt fails, then another node must have been inserted at the head of the list after this

operation read the head pointer. In this case, the operation retries by reading the head pointer again and updating the new node's prev pointer accordingly. When the store() operation eventually succeeds, it moves on to Line 11 where it attempts to retrieve and set the timestamp of the node. By Lemma 1, this operation is wait-free, and upon completion, the node is guaranteed to have a valid timestamp. This timestamp is then returned at Line 12. Thus, each store() invocation returns the timestamp that is installed in the node it inserts into the list.

Theorem 1. The ordering of timestamps returned by store operations is consistent with the ordering of the corresponding effective store operations.

Proof. A new node created by a store() operation is inserted into the list at the successful iteration of Line 10. This node initially has timestamp TBD. By Lemma 1, the node has a valid timestamp upon completion of the help_timestamp() invocation at Line 11. If the linearization point is as stated in the theorem above, it clearly lies between these two lines and thus, within the invocation and response of a store() operation.

By Lemmas 4 and 5, the nodes in the list are totally ordered by their insertion times, and the nodes' timestamps and corresponding get_timestamp() invocations reflect this ordering. The get_timestamp() operation consists of a single atomic fetch_add instruction on a global timestamp. By Lemma 6, the timestamps stored in the nodes are identical to those returned by their corresponding store() operations. Since fetch_add returns a unique increasing value for each invocation, the get_timestamp() invocations induce a total order over the corresponding store() operations. Consequently, the store() operations can be ordered according to the timestamps they return. This ordering is consistent with the structure of the list, where the head node corresponds to the most recent store() operation. Therefore, ordering store() operations by the fetch_add instruction within their corresponding get_timestamp() invocations yields a total order that respects the real-time ordering of store() operations. Thus, the store() operations are linearizable. ∎

Theorem 2. The load() operation returns the node with a timestamp equivalent to at most the timestamp it is linearized at.

Proof. According to Section 2 of this document, the store() operation is linearized at the get_timestamp() invocation that sets the timestamp of the newly inserted node. In contrast, the load() operation is linearized at the point it retrieves the global timestamp via get_timestamp() at Line 3. Let this timestamp be T. It then retrieves the head pointer at Line 4 and assists in setting the timestamp of the head node at Line 5. By Lemma 1, the head node has a valid timestamp after Line 5. By Lemma 2, all nodes in the list now have valid timestamps. Thus, curr -> timestamp.load() in the while loop condition at Line 7 will never be equal to TBD.

By Lemma 6, each node is assigned the timestamp returned by its corresponding invocation of store(). By Lemma 5, the nodes in the list have strictly decreasing timestamps.

Therefore, the timestamps establish a total order on the inserted nodes that matches the order of their corresponding store() operations. By Lemma 3, no two timestamps can be identical. Thus, the most recent store() operation linearized before the load() operation must return a timestamp strictly less than T. Consequently, every node inserted by a store() operation linearized before this load() operation has a timestamp less than T. Between reading the global timestamp at Line 3 and reading the head pointer at Line 4, one or more nodes may be inserted into the list. These nodes are inserted into the list after Line 3, which is the linearization point of the load() operation. Therefore, their get_timestamp() invocations occur strictly after the get_timestamp() invocation of this load() in real time, and they consequently obtain timestamps strictly greater than T. The while loop effectively skips these nodes by repeatedly checking the condition curr -> timestamp.load() > read_ts and returns the node with the largest timestamp less than T. At the time get_timestamp() returned T at Line 3, this node was either the head node, if it had a valid timestamp, or the node immediately following the head node otherwise. Thus, load() correctly returns the node that was the effective head node at its linearization point. ∎

Theorem 3. Any possible interleaving of load and store operations produces a linearizable history.

Proof. Every load() and store() operation can be linearized at its invocation of get_timestamp(). More specifically, at the linearization point of get_timestamp() which is the fetch_add instruction of the global timestamp. By Lemma 3, these timestamps are unique. Consequently, these operations can be totally ordered according to the timestamps returned by their respective get_timestamp() invocations. Moreover, the operations can be regarded as taking effect at these linearization points, so that overlapping invocations appear to execute according to a sequential history of load() and store() operations that satisfy their sequential specification. ∎

Theorem 4. Any possible interleaving of load, store, and CAS operations produces a linearizable history.

Proof. The CAS() operation has two distinct cases of execution. It returns false at Line 15 with the global timestamp read at Line 5 or returns true at Line 22 with the timestamp installed in the newly created node. We must prove that in both cases, the CAS() operation is linearizable.

In the case that the operation returns false, its linearization point is the read of the global timestamp at Line 5. The head pointer is read subsequently at Line 6. This pointer may no longer point to the head node at the linearization point (the read of the global timestamp at Line 5), since one or more nodes may have been inserted between Lines 5 and 6. Using the global timestamp read at Line 5, the while loop traverses the list, analogously to load(), to locate the head node at the linearization point. Since the operation correctly identifies the head node at the linearization point, it effectively reads the corresponding value and verifies that it does not match the expected value at that point. By

Lemma 3, all store() operations that are possibly linearized after this point return timestamps strictly larger than the timestamp read by the CAS() operation. Consequently, the CAS() operation does not consider any nodes inserted by these store() operations, as enforced by the traversal condition at Line 9. Although the read of the global timestamp is performed inside an outer while(true) loop, the mismatch between the expected value and the current value in the failure case ensures that the loop does not repeat. Hence, the outer loop can be disregarded in this scenario.

In the case that the operation passes the condition at Line 13 but fails to insert the newly created node, it is linearized at the read of the global timestamp at Line 5 in the last iteration of the outer loop. This corresponds to the iteration in which the expected value differs from the value observed at the linearization point. Similar to the previous case, the operation is linearized at the read of the global timestamp, where it takes effect in detecting the mismatch. The linearization point is shifted to the last iteration because all previous iterations observed matching values. If the operation were linearized at any of those earlier points, it should have returned true, which it did not due to contention. Since the operation ultimately returns false, it must be linearized at the point where it reads the node exhibiting the mismatch.

In the final case, where the operation returns true, its linearization point is the fetch_add instruction in the get_timestamp() call within the successful help_timestamp() invocation for the newly inserted node. As mentioned in previous cases, the node read at Line 5, curr, is the head node at the time the operation reads the global timestamp at Line 5. Analogous to store() operations, due to the compare_exchange_weak instruction at Line 20, at most one thread can successfully update the shared head pointer at any given time. Thus, at most one node can be inserted into the shared list at a time. The successful execution of this instruction also ensures that no concurrent insertion has modified the head node to a value different from expected, since any such node would not be equivalent to curr read at Line 6. Even if another node appended after Line 6 coincidentally has the same value as expected, the compare_exchange_weak at Line 20 still fails. Thus, any modification of the shared list causes another iteration of the outer loop. Most importantly, execution of the if-clause at Line 20 implies that there is no node between the newly inserted node with the desired value and the node with the matching value. By Lemma 1, the help_timestamp() invocation at Line 21 installs the timestamp for the new node in constant time, after which the node becomes visible to any thread performing load(), store(), or CAS() operations.

All concurrent store() and modifying CAS() operations can insert at most one node into the shared list due to the compare_exchange_weak instruction at Lines 10 and 20, respectively. By Lemma 2, the timestamp for this node must be installed before any other node can be inserted, since there cannot be two nodes with timestamp TBD at any given instant. Consequently, node insertions are totally ordered, and thus the get_timestamp() calls within the corresponding help_timestamp() calls are likewise totally ordered. By Lemma 3, the timestamps returned by these calls are strictly increasing. Therefore, overlapping store() and modifying CAS() operations can be totally ordered according to the order in which they insert nodes, by the timestamps they return. Thus, CAS() operations are linearizable in both unsuccessful and modifying cases. ∎

4. References

[1] Yuanhao Wei, Naama Ben-David, Guy E. Blelloch, Panagiota Fatourou, Eric Ruppert, & Yihan Sun. 2020. *Constant-Time Snapshots with Applications to Concurrent Data Structures.*

[2] Maurice Herlihy and Nir Shavit. *The Art of Multiprocessor Programming.* Morgan Kaufmann, 2012.

The following work is draft material.

Alternative get_timestamp functions

```
1 int get_timestamp() {
2     return timestamp.fetch_add(1);
3 }
```

This version guarantees unique timestamps since we are using the fetch_add which ensures that read-modify-write is indivisible. We later prove that load(), store(), and CAS() are all linearized at get_timestamp calls and by the use of this version, they return strictly increasing timestamps.

```
1 int get_timestamp() {
2     return __builtin_ia32_rdtsc();
3 }
```

This version reads the TSC (Time Stamp Counter), accessible in x86 via the RDTSC instruction. It is a high-resolution counter that increments at a constant rate and is shared and consistent across all cores. However, although this occurs rarely, multiple threads may read the counter simultaneously, resulting in identical timestamps.

Proof of Correctness

In this section, we prove that the atomic timestamping algorithm is linearizable. We first provide some lemmas that will aid in our formal proof of linearizability.

A timestamp is considered valid if it is not TBD; otherwise, it is considered invalid.

Lemma 1. After an execution of help_timestamp on a given node, that node has a valid timestamp. Furthermore, help_timestamp is a wait-free operation.

Proof. The help_timestamp() function enters the if clause when the given node has an invalid timestamp. Therefore, if execution does not enter the if clause, the node must have a valid timestamp.If execution enters the if clause and the CAS succeeds, the current thread successfully updates the timestamp to a valid value. If the CAS fails, the current value of the timestamp must differ from TBD, implying that another thread has already set the timestamp to a valid value. Thus, in all executions of help_timestamp(), the given node has a valid timestamp upon termination, and the function completes in a constant number of instructions. Moreover, a node's timestamp is modified only within the help_timestamp

function. Since this function updates the timestamp only if it is TBD, a valid timestamp assigned by the successful help_timestamp invocation remains unchanged.

Lemma 2. The only node in the TimestampedAtomic object's linked list that can have an invalid timestamp is the very first node of the list.

Proof. Since the implementation of load(), store(), and CAS() do not attempt to remove any nodes from the list, no node is discarded or manipulated after it is appended to the list. Therefore, the only case where the list is changed is when we have a new node appended to the head of the list. This change can only be produced by store() and modifying CAS() operations. Both operations successfully append a new node after the successful execution of the CAS on the head pointer at Lines 10 and 18, respectively. Both operations call help_timestamp() on the current head node before the CAS on Line 6. By Lemma 1, the head node has a valid timestamp after Line 6. Therefore, by induction, every newly appended node is added to a list consisting entirely of nodes with valid timestamps.

Lemma 3. get_timestamp() returns timestamps in non-decreasing order.

Proof. Every TimestampedAtomic object is associated with a single shared Clock object, which contains a single atomic integer field, timestamp. Since the only operation that modifies this field is get_timestamp() which increments it atomically, sequential invocations of get_timestamp() return strictly increasing timestamps. Concurrent invocations result in exactly one thread successfully incrementing the timestamp via CAS. All other threads fail and return the value written by the successful thread. Thus, the returned values are non-decreasing.

Theorem 1. load() and store() operations that return timestamps are linearizable and return timestamps in non-decreasing order.

Proof. According to the linearization points specified in Section 2 of this document, all load() and store() operations are linearized immediately after the call to get_timestamp(). By Lemma 3, the timestamps returned by load() and store() are non-decreasing at the time each operation invokes get_timestamp(). Therefore, if we order the operations according to their timestamps, it would be equivalent to the linearization point order.

Since operations may return timestamps with the same value, we must prove that these operations are linearizable in this case. load() operations that return the same timestamp can be reordered in any way since the ordering of these operations do not manipulate the list. The correctness of load operations is discussed in the proof of Theorem 2. Therefore, it suffices to show that

1. load() and store() operations returning the same timestamp
2. overlapping store() operations returning the same timestamp

can be consistently ordered in a sequential linearization trace.

Suppose threads concurrently executing load() and store() operations return the same timestamp T in their respective get_timestamp() invocations. If all threads performing store() succeed in setting the timestamp of their newly created nodes before any thread performing load() reads the head pointer at Line 4, then the threads performing load() will observe that the head node has a valid timestamp of T and will return that node. Thus, load() operations correctly return the node with timestamp at most T. In this case, load operations can be linearized after store operations.

The only remaining case is when at least one thread performing load() reads the head pointer before some thread performing store() has finished installing timestamp T in its newly created node. By Lemma 2, the head node is the only node that can have an invalid timestamp. Therefore, there can be at most one thread that has not installed timestamp T for its newly created node, given that every other thread succeeds in installing timestamp T for their respective nodes. Threads executing load() that observe the head node having an invalid timestamp will attempt to set it at help_timestamp() at Line 5. These threads will call get_timestamp() in help_timestamp(). All values returned by these get_timestamp() invocations are strictly larger than T, since the corresponding calls occur later in program order than the get_timestamp() calls at Line 3. One of these threads will succeed in setting the timestamp for the head node and the store() operation will be linearized to the get_timestamp() invocation of the succeeding thread. Thus, the store operation is linearized after all the load operations with a timestamp larger than T. Hence, we have reached a contradiction with our assumption that the store() operation returns the same timestamp as load() operations.

Consequently, when load() and store() operations return the same timestamp T, the only possible case is that all store() threads successfully set the timestamp of their newly created nodes before any load() thread reads the head pointer. In this scenario, the load() threads correctly observe the nodes inserted by the store() threads, consistent with the specification that load() returns a node with timestamp at most T. Furthermore, whenever load() and store() operations return the same timestamp, all load() operations can be consistently linearized after all store() operations.

Consider the case where there are multiple concurrent store() operations that each return the same timestamp T at get_timestamp() in help_timestamp() at Line 11. To have reached Line 11, it implies that threads concurrently passed the if condition at Line 10. Since this CAS is an atomic operation on the head pointer, two or more threads cannot concurrently pass the if condition. Thus, all other threads fail the CAS and reload the head pointer at Line 5. The timestamp of the newly inserted node is set at Line 11 if the inserting thread successfully installs it, or at Line 6 if another thread performs the update. Then, another node is inserted with its timestamp set by a subsequent help_timestamp() invocation either at Line 6 or Line 11. Since nodes are inserted sequentially and the timestamp for that node is installed before another node can be inserted into the list, no two nodes can have the same timestamp. Moreover, timestamps are installed in nodes in strictly increasing order which are the timestamps returned by store() operations. Hence, we have reached a contradiction with our assumption that concurrent store() operations return the same timestamp. This gives rise to another valuable lemma.

Consider the case where concurrent store() operations on two different TimestampedAtomic objects return identical timestamps. Although it may appear that these operations can be reordered arbitrarily, the reads-from, program-order, and coherence-order relations may impose a strict ordering on these concurrent store() operations. Consider the following example where threads A and B performing store() return identical timestamps ts+1. The two different TimestampedAtomic objects are abbreviated as TA1 and TA2.

Timestamp	A	B
ts		TA1.load() -> x
ts+1	TA1.store(x)	TA2.store(y)

Proof.

Lemma 4. The list maintains a strictly decreasing order of timestamps at all times, with the possible exception of the head node that may have timestamp TBD**.**

Next, we provide the formal proof of correctness for load().

Theorem 2. load() returns a node whose timestamp is at most the timestamp which load() is linearized at.

Proof. According to Section 2 of this document, load() is linearized at the point it retrieves the global timestamp via get_timestamp() at Line 3. Let this timestamp be T. It then retrieves the head pointer at Line 4 and assists in setting the timestamp of the head node at Line 5. By Lemma 1, the head node has a valid timestamp after Line 5. By Lemma 2, all nodes in the list have valid timestamps.

By Theorem 1, all store() operations that are linearized before the load() operation returns timestamps less than or equal to T. Each newly inserted node receives its timestamp through a successful CAS operation in help_timestamp, performed by an arbitrary thread. By Lemma 1, each node has a valid timestamp after the invocation of help_timestamp at Line 11 of the store() operation that inserted the node. The assigned timestamp is subsequently read at Line 12 and immediately returned. Hence, each node is assigned the timestamp returned by its corresponding invocation of store(). By the proof of Theorem 1, no two store() operations can return the same timestamp. Therefore, starting from the head node read by load() at Line 4, the nodes in the list have strictly decreasing timestamps, each of which is less than or equal to T.

While traversing the list, curr -> timestamp.load() will never return TBD. Since load() skips all nodes with timestamp >T in the while loop, and the list has strictly decreasing timestamps by Lemma 4, it returns the first node whose timestamp is at most T. Since it is the first node with timestamp ≤ T, it was the head node when get_timestamp() returned T at Line 3. Thus, load() correctly returns the node that was the head node at its linearization point.

The correctness of failing CAS() and successful CAS() can be proved analogously to the proofs for load() and store(), respectively.

A: load() B: store()

Case 1:

A: int ts = get_timestamp() <- returns timestamp t

B: get_timestamp() in help_timestamp at Line 19 <- returns timestamp t

B: finishes CAS of timestamp t for newly added node N in help_timestamp()

A: reads N

Case 2:

A: int ts = get_timestamp() <- returns timestamp t

B: get_timestamp() in help_timestamp at Line 19 <- returns timestamp t

A: read head on Line 4

A: help_timestamp(head) on Line 5

A: get_timestamp() in help_timestamp() returns timestamp k, which is larger than t

A: finishes CAS of timestamp k for newly added node N in help_timestamp(head)

A: does not read N

3-1. General Outline

load(): takeSnapshot() + readSnapshot(int t)

prove that load() returns the last value to be added with timestamp t before timestamp changes from t to t+1.

get_timestamp() gives us a timestamp t at Line 3.

Since head is valid after help_timestamp(head) at Line 5, all nodes in the list are valid starting from head. <- We need a Lemma that states that the only node that can be invalid is the head.

curr -> timestamp is always valid in the loop condition at Line 7.

Since we descend down the list according to timestamps, the node we found is the latest node in the list with timestamp t <- the loop condition is strictly increasing

All nodes that we've traversed have timestamps strictly larger than t.

Since we t = get_timestamp() at Line 3 and read the head at Line 4, the head and all other nodes preceding it afterwards have timestamps larger than t.

There is no node between setting timestamp t to t+1 and the node retrieved by load().

Thus, the load operation returns the latest node with timestamp t.

store(): Consider s to be the linearization point of store() which is during the successful CAS operation of the head pointer. More specifically, when we retrieve the global timestamp for the new node in help_timestamp(newNode) at Line 11.

Before s but when the new node has successfully appended to the list, the linearization point of store() has not occurred yet. So, head-> prev -> val is the value of the list.

After s, there could be a window where the new node has no valid timestamp???

After s when the timestamp is set, the value of the list is head -> val.

Theorem 1. *In every configuration C after the constructor of the version list has completed,*

1. *if head points to the Node created by the constructor of the version list, then head->val is the value of the timestamped object,*
2. *if the linearization point of the store() or CAS() that created the first node in the version list is before C, then head->val is the value of the the timestamped object, and*
3. *otherwise, head->prev->val is the value of the timestamped object.*

So, it follows this theorem. How does this theorem prove that overlapping write operations are correct???